第4章: React Native / Expo入門

> 第3章で学んだReactの考え方を、いよいよ**スマホアプリの画面**に応用します。この章では、React Nativeの基本部品（ `View` / `Text` / `Image` など）、画面の配置を決める **Flexbox**、そして画面の切り替えを担う **Expo Router** を、完全初心者向けに解説します。

---

1. React Native のコア部品

第3章の最後で触れたとおり、スマホアプリにはブラウザが無いので、HTMLタグ（ `<div>` , `<p>` ）は使えません。代わりにReact Nativeが用意した**専用部品（コアコンポーネント）** を使います。

1.1 主要部品の対応表

役割	WebのHTML	React Native	ひとこと
まとまり・箱	`<div>`	**`<View>`**	レイアウトの基本。すべての土台
文字の表示	`<p>` , `<span>`	**`<Text>`**	**文字は必ずこれで囲む**（重要ルール）
画像	`<img>`	**`<Image>`**	画像の表示
入力欄	`<input>`	**`<TextInput>`**	キーボード入力を受け取る
押せるもの	`<button>`	**`<Pressable>`** / `<Button>`	タップ操作を受け取る
縦スクロール	（自動）	**`<ScrollView>`** / `<FlatList>`	スマホは画面が小さいので必須

> **最重要ルール — 文字は必ず `<Text>` で囲む:** Webでは `<div>こんにちは</div>` のように文字を直接置けますが、**React Nativeでは文字（テキスト）は必ず `<Text>` の中に入れなければエラー**になります。「 `<View>` の中に文字を直接書いてはいけない、 `<Text>` で包む」と覚えてください。これは初心者が最初に必ずつまずくポイントです。

1.2 `View` と `Text` — 最小のアプリ

第1章で作った `my-books-app` の `app/(tabs)/index.tsx` を、次の内容に書き換えて試してみましょう（既存の内容を消して貼り付け）。

> **▼ このコードがやること（先に日本語で）**：画面の中央に「📚 書籍管理アプリ」という太字の文字を1つ表示する、いちばん小さなアプリを作ります。ポイントは2つ——①箱（まとまり）には `<View>` を使い、文字は必ず `<Text>` で囲むこと、② `style` に「中央寄せ」の指定を渡して位置を決めることです。細かな各指定の意味はコード内のコメントで説明しています。

コピー

```
// react-native という部品集から View と Text を借りてくる
import { View, Text } from "react-native";

// 画面のコンポーネント。export default で「この画面を公開」する
export default function HomeScreen() {
  return (
    // <View> : 箱。style で見た目を指定（後述のFlexbox）
    <View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
      {/* flex:1 → 画面いっぱいに広がる / justifyContent:"center" → 縦方向に中央 / alignItems:"center" → 横方向に中央 */}
      <Text style={{ fontSize: 24, fontWeight: "bold" }}>📚 書籍管理アプリ</Text>
      {/* fontSize:24 → 文字サイズ24 / fontWeight:"bold" → 太字 / 文字は必ず<Text>で囲む */}
    </View>
  );
}
```

> **`style={{ ... }}` の中カッコが2つあるのはなぜ？** 外側の `{ }` は「JSXの中にTypeScriptを書く」ための中カッコ、内側の `{ }` は「スタイルを表すオブジェクト（第2章のオブジェクト）」です。だから `style={{ fontSize: 24 }}` は「styleにオブジェクト `{fontSize: 24}` を渡す」という意味になります。最初は2重に見えて戸惑いますが、慣れれば自然です。
>
> **Webのstyleとの違い:** Webの `font-size` はReact Nativeでは `fontSize`（キャメルケース）、値も `"24px"` ではなく数値の `24` です。区切りのハイフンを無くして次の単語を大文字にする「キャメルケース」で書く、と覚えてください。

▼ コードを1つずつ分解して解説

上のコードを、初心者がつまずきやすい順に**1つずつ**ていねいに見ていきましょう。

解説1: 部品（View・Text）を借りてくる `import`

コピー

```
import { View, Text } from "react-native";
```

- `import`（インポート＝取り込む）は、「**別の場所で用意された部品を、このファイルに借りてくる**」命令です。
- `{ View, Text }` の `{ }` は「その中の名前のものだけを借りる」という意味で、ここでは `View`（箱）と `Text`（文字）の2つを借りています。
- `from "react-native"` は「`react-native` **という部品集（ライブラリ）から借りる**」という意味です。React Native本体に最初から入っている部品なので、追加インストールは不要です。

- HTMLの `<div>` / `<p>` と違い、React Nativeの部品は**使う前に必ずこの `import` で借りてくる**必要があります。借り忘れると「`View` が見つからない」というエラーになります。

> **用語: ライブラリ**とは「よく使う部品や機能を、誰かがまとめて用意してくれた道具箱」のこと。`react-native` はスマホアプリ用の部品が詰まった道具箱で、そこから必要な部品名を指定して取り出すのが `import` です。

解説2: 画面を公開する `export default function`

コピー

```
export default function HomeScreen() {
  return (
```

- `function HomeScreen()` は「`HomeScreen` という名前のコンポーネント（画面の部品）を作る」という宣言です。コンポーネント名は**大文字始まり**にするのがReactの決まりです。
- `export default` （エクスポート・デフォルト）は「**この画面を、ファイルの外から使えるように公開する**」という印です。Expo Routerは「ファイルが `default` で公開している部品」を、その画面の中身として表示します。
- `return ( ... )` は「**この画面が表示する見た目（JSX）を返す**」部分です。`( )` で囲むことで、複数行のJSXを見やすく書けます。

> **用語: `export default`** は「このファイルの代表（主役）はコレですよ」と1つだけ指定する書き方です。Expo Routerでは、各画面ファイルがこの形で1つの画面を公開する、と覚えてください。

解説3: 箱と中央寄せ — `<View style={{...}}>`

コピー

```
<View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
```

- `<View>` は、Webの `<div>` にあたる「**まとまり（箱）**」です。中に文字や他の部品を入れ、位置やサイズをここで決めます。
- `style={{ ... }}` の**外側の `{ }`** は「JSXの中にTypeScriptを書く印」、**内側の `{ }`** は「スタイルを表すオブジェクト」です（中カッコが2重になる理由）。
- `flex: 1` は「**使える空間を全部もらって広がる**」指定。一番外側の箱に付けると画面全体に広がります。
- `justifyContent: "center"` と `alignItems: "center"` の2つで、中身を**縦・横ともに中央**に寄せています（詳しい向きの話は次の2章のFlexboxで解説）。

> **用語: `flex: 1`** は「余っている空間を全部使って広がれ」という命令。画面全体に広げたい一番外側のViewによく付けます。

解説4: 文字は必ず `<Text>` で囲む

コピー

```
<Text style={{ fontSize: 24, fontWeight: "bold" }}📚 書籍管理アプリ</Text>
```

- React Nativeでは、**文字（テキスト）は必ず `<Text>` の中に入れる**のが絶対ルールです。`<View>` に文字を直接書くとエラーになります。
- `fontSize: 24` は文字サイズを24に。Webと違い `"24px"` ではなく**数値の `24`** で書きます。
- `fontWeight: "bold"` は太字にする指定です。`bold`（ボールド）は「太字」を意味する英単語です。
- `<Text>` と `</Text>` で挟まれた `📚 書籍管理アプリ` の部分が、実際に画面に表示される文字です。

> **用語: `<Text>`** はReact Native専用の「文字を表示する部品」。Webの `<p>` / `<span>` にあたります。「文字を見たら `<Text>` で囲む」が初心者の鉄則です。

2. Flexbox — 部品を画面に配置する仕組み

スマホ画面に部品を「中央寄せ」「横並び」「均等配置」する仕組みが **Flexbox（フレックスボックス）** です。React Nativeのレイアウトは基本すべてこのFlexboxで行います。

2.1 主要なプロパティ

> **▼ このコードがやること（先に日本語で）:** 「左・中・右」の3つの文字を、箱（ `<View>` ）の中に**横並び**で配置し、両端と中央にバランスよく散らす例です。`flexDirection` で「並べる向き」、`justifyContent` と `alignItems` で「並べたものの寄せ方」を決めます。まずは「向き」と「寄せ方」を別々に指定するのだ、という感覚をつかんでください。各値の意味はコード内のコメントにあります。

コピー

```
<View
  style={{
    // 子要素を並べる方向。"row"=横並び / "column"=縦並び（既定）
    flexDirection: "row",
    // 「主軸方向」の配置。下の表参照
    justifyContent: "space-between",
    // 「交差軸方向」の配置。下の表参照
    alignItems: "center",
    // 子要素同士の間隔（隙間）を8に
    gap: 8,
  }}
>
  <Text>左</Text>
  <Text>中</Text>
  <Text>右</Text>
</View>
```

▼ コードを1つずつ分解して解説

このViewのstyleに書いた4つの指定を、1つずつ見ていきましょう。

解説1: 並べる向きを決める `flexDirection`

コピー

```
// 子要素を並べる方向。"row"=横並び / "column"=縦並び（既定）
flexDirection: "row",
```

- `flexDirection`（フレックス・ディレクション＝並べる向き）は、「**箱の中の子要素を、どの向きに並べるか**」を決めます。
- `"row"`（ロウ＝行）にすると**横並び**になります。何も指定しないと既定は `"column"`（カラム＝列）で**縦並び**です。
- ここでは `"row"` なので「左・中・右」の3つの文字が**横一列**に並びます。

> **用語: 主軸（main axis）** とは、`flexDirection` で決めた「並べている向き」のこと。`"row"` なら主軸は横方向、`"column"` なら主軸は縦方向になります。

解説2: 主軸方向の寄せ方 `justifyContent`

コピー

```
// 「主軸方向」の配置。下の表参照
justifyContent: "space-between",
```

- `justifyContent`（ジャスティファイ・コンテント）は、「**並べている向き（主軸）に沿って、子要素をどう寄せるか**」を決めます。
- `"space-between"`（スペース・ビトウィーン）は「**両端の子を端いっぱいに置き、間に均等な隙間を空ける**」配置です。ここでは「左」が左端、「右」が右端、「中」がその真ん中に来ます。
- 他に `"center"`（中央に寄せる）や `"flex-start"`（先頭に寄せる）などがあります。

> **用語: `justifyContent`** は「主軸（並べている向き）の寄せ方」。`flexDirection: "row"` のときは横方向の寄せ方になります。

解説3: 直角方向の寄せ方 `alignItems`

コピー

```
// 「交差軸方向」の配置。下の表参照
alignItems: "center",
```

- `alignItems`（アライン・アイテムズ）は、「**並べている向きと直角の方向に、子要素をどう寄せるか**」を決めます。
- `flexDirection: "row"`（横並び）のときは、直角方向＝**縦方向**なので、`"center"` で子が**縦の中央**にそろいます。
- `justifyContent` と効く向きが直角になっている点が、最初に混乱しやすいポイントです。

> **用語: 交差軸（cross axis）** とは、主軸に対して直角の向きのこと。`alignItems` はこの交差軸方向の寄せ方を決めます。

解説4: 子要素同士の隙間 `gap`

コピー

```
// 子要素同士の間隔（隙間）を8に
gap: 8,
```

- `gap`（ギャップ＝隙間）は、「**並べた子要素と子要素の間に空ける隙間**」の大きさです。
- ここでは `8` なので、各文字の間に8ぶんの隙間ができます。値は数値で書きます（Webのような `"8px"` ではない）。

- 昔は隙間を作るのに `margin` を使っていましたが、`gap` を使うと「子の間だけ均等に空ける」が簡単に書けて便利です。

> **用語: `gap`** は「子要素どうしの間隔」専用の指定。外側の余白を作る `padding`（内側）や `margin`（外側）とは役割が違います。

プロパティ	役割	よく使う値
`flexDirection`	子要素を並べる向き	`"row"`（横）/ `"column"`（縦・既定）
`justifyContent`	並べる方向の配置	`"center"`（中央）/ `"space-between"`（両端に寄せ間を空ける）/ `"flex-start"`（先頭寄せ）
`alignItems`	直角方向の配置	`"center"`（中央）/ `"stretch"`（伸ばす）/ `"flex-start"`
`flex`	余白をどう分け合うか	`1`（利用可能なスペースいっぱいに広がる）
`gap`	子要素同士の隙間	`8`, `12` など数値
`padding`	自分の内側の余白	`16` など数値
`margin`	自分の外側の余白	`8` など数値

> **`justifyContent` と `alignItems` がややこしい理由:** この2つは `flexDirection` の向きによって「効く方向」が入れ替わります。- `flexDirection: "column"`（縦並び・既定）のとき → `justifyContent` は**縦**方向、`alignItems` は**横**方向 - `flexDirection: "row"`（横並び）のとき → `justifyContent` は**横**方向、`alignItems` は**縦**方向 最初は混乱しますが、「`justifyContent` ＝並べてる向きの配置」「`alignItems` ＝それと直角の配置」と覚え、実際に値を変えて画面の変化を見ながら慣れるのが一番です。
>
> **`flex: 1` の意味:** 「使える空間を全部もらって広がる」という指定です。一番外側の `<View>` に `flex: 1` を付けると、その箱が画面全体に広がります。複数の子に `flex: 1` を付けると、空間を等分します。

2.2 StyleSheet で書く（推奨）

スタイルを `style={{ }}` の形で直接書くと、コードが読みにくくなります。React Nativeには `StyleSheet` という、スタイルを別にまとめて名前で呼ぶ仕組みがあります。

▼ **このコードがやること（先に日本語で）**：さきほどと同じ「中央に太字の見出し」を、今度は**スタイルを別の場所にまとめて**書く形に整理します。`StyleSheet.create({ ... })` でスタイルに `container` や `title` といった名前を付けておき、JSX側では `styles.container` と名前で呼び出します。「見た目の指定」と「画面の構造」を分けると読みやすくなる、という整理術がポイントです。詳細はコード内コメントを参照してください。

コピー

```
// StyleSheet も借りる
import { View, Text, StyleSheet } from "react-native";

export default function HomeScreen() {
  return (
    {/* styles.container で下の定義を参照 */}
    <View style={styles.container}>
      <Text style={styles.title}>📚 書籍管理アプリ</Text>
    </View>
  );
}

// StyleSheet.create({ ... }) : スタイルをまとめて定義する。名前を付けて再利用できる
const styles = StyleSheet.create({
  // container という名前のスタイル
  container: {
    // 画面いっぱい
    flex: 1,
    // 縦中央
    justifyContent: "center",
    // 横中央
    alignItems: "center",
    // 背景色を白に（#fff は白を表す色コード）
    backgroundColor: "#fff",
  },
  // title という名前のスタイル
  title: {
    fontSize: 24,
    fontWeight: "bold",
    // 文字色（#1e293b は濃いグレー）
    color: "#1e293b",
  },
});
```

`StyleSheet.create` を使うメリット: ①画面の構造（JSX）とスタイルが分離されて読みやすい ②同じスタイルを使い回せる ③タイプミスを早く発見できる。第9章では、これに加えて **NativeWind**（Tailwind風の書き方）も導入し、さらに楽にスタイリングします。

▼ コードを1つずつ分解して解説

「スタイルを別にまとめて名前で呼ぶ」という新しい書き方を、1つずつ見ていきましょう。

解説1: 名前でスタイルを呼ぶ `styles.container`

コピー

```
{/* styles.container で下の定義を参照 */}
<View style={styles.container}>
  <Text style={styles.title}>📚 書籍管理アプリ</Text>
</View>
```

- 直接 `style={{ ... }}` と書く代わりに、`style={styles.container}` のように「**スタイルの名前**」を渡しています。
- `styles` は下のほうで作る「スタイルの一覧（まとめ）」で、`styles.container` は「その中の `container` という名前のスタイル」を指します（`.` でその中の項目を取り出す書き方）。
- こうすると、JSX側には「どのスタイルを使うか」だけが書かれ、**見た目の詳しい指定はすべて下にまとまる**ので、画面の構造が読みやすくなります。

> **用語: `styles.container` の `.`**（ドット）は「オブジェクトの中の項目を取り出す」記号。`styles` という箱の中の `container` を取り出している、という意味です。

解説2: スタイルをまとめて定義する `StyleSheet.create`

コピー

```
const styles = StyleSheet.create({
  // container という名前のスタイル
  container: {
    // 画面いっぱい
    flex: 1,
    // 縦中央
    justifyContent: "center",
    // 横中央
    alignItems: "center",
    // 背景色を白に（#fff は白を表す色コード）
    backgroundColor: "#fff",
  },
  // title という名前のスタイル
  title: {
    fontSize: 24,
    fontWeight: "bold",
    // 文字色（#1e293b は濃いグレー）
    color: "#1e293b",
  },
});
```

- `StyleSheet.create({ ... })` は「**複数のスタイルに名前を付けてまとめる**」道具です。結果を `styles` という変数に入れて、JSX側から `styles.名前` で呼び出します。
- `{ ... }` の中に `container:` と `title:` という2つの名前を作り、それぞれに `{ }` でスタイルの中身を書いています。名前は自由に付けられます。
- `backgroundColor: "#fff"` の `#fff` は「色を表すコード（カラーコード）」で、`#fff` は白を意味します。`#1e293b` は濃いグレーです。
- `const` で宣言しているのは「この `styles` は後で書き換えない（固定）」という意味です。

> **用語: カラーコード**とは、色を `#` で始まる英数字で表す書き方。`#fff`（白）や `#1e293b`（濃いグレー）のように、Webでもアプリでも共通で使えます。

---

3. ユーザー操作を受け取る部品

3.1 `TextInput` — 文字入力

検索ボックスや、本のタイトルを入力するフォームに使います。

▼ **このコードがやること（先に日本語で）**：文字を打ち込める入力欄を作り、**打った文字をその場で画面に表示する**例です。入力中の文字は `useState`（第3章で学んだ「状態」）にしまっておき、入力が変わるたびに `onChangeText` でその状態を更新します。「入力欄の中身＝状態」という結びつきを作るのがコツで、これはフォームの基本形です。各指定の意味はコード内コメントにあります。

コピー

```
import { useState } from "react";
import { View, TextInput, Text, StyleSheet } from "react-native";

export default function SearchBox() {
  // 入力文字をstateで管理（最初は空文字""）
  const [keyword, setKeyword] = useState("");

  return (
    <View>
      <TextInput
        // 見た目
        style={styles.input}
        // placeholder ： 未入力時に薄く表示する案内文
        placeholder="タイトルや著者で検索..."
        // value ： 表示する値。stateと結びつける
        value={keyword}
        // onChangeText ： 入力が変わるたびに呼ばれる。textは今の入力内容
        onChangeText={(text) => setKeyword(text)}
      />
      {/* 入力した文字がリアルタイムで表示される */}
      <Text>入力中: {keyword}</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  input: {
    // 枠線の太さ1
    borderWidth: 1,
    // 枠線の色（薄いグレー）
    borderColor: "#e2e8f0",
    // 角の丸み8
    borderRadius: 8,
    // 内側の余白
    padding: 10,
  },
});
```

> **`value` と `onChangeText` のセット**: この2つを組み合わせると「stateが入力欄の中身を管理し、入力が変わるとstateも更新される」という双方向の結びつきができます。これを「制御コンポーネント（controlled component）」と呼びます。フォームの基本パターンなので、第7章でも使います。

▼ コードを1つずつ分解して解説

「入力欄の中身＝状態」という結びつきを作る部分を、1つずつ見ていきましょう。

解説1: 入力内容を覚えておく `useState`

コピー

```
// 入力文字をstateで管理（最初は空文字""）
const [keyword, setKeyword] = useState("");
```

- `useState`（ユーズ・ステート）は、第3章で学んだ「**変化する値を持つための道具**」です。`const [今の値, 値を変える関数] = useState(初期値)` の形で使います。
- ここでは「今の値」を `keyword`（入力されている文字）、「値を変える関数」を `setKeyword` という名前にしています。
- `useState("")` の `""` は初期値で、**最初は空っぽの文字**から始まる、という意味です。
- 入力欄に打った文字をこの `keyword` にしまっておくことで、「今なにが入力されているか」をアプリが覚えていられます。

> **用語: state（ステート＝状態）** とは「変化して、変わると画面も更新される値」のこと。`useState` でその状態の入れ物と、それを更新する関数をセットで作ります。

解説2: 入力欄を状態に結びつける `value` と `onChangeText`

コピー

```
<TextInput
  // 見た目
  style={styles.input}
  // placeholder : 未入力時に薄く表示する案内文
  placeholder="タイトルや著者で検索..."
  // value : 表示する値。stateと結びつける
  value={keyword}
  // onChangeText : 入力が変わるたびに呼ばれる。textは今の入力内容
  onChangeText={(text) => setKeyword(text)}
/>
```

- `<TextInput>` は、Webの `<input>` にあたる「**文字を打ち込める入力欄**」の部品です。

- `placeholder`（プレースホルダー）は「未入力のときに薄い文字で表示する案内文」。ヒントを出すだけで、実際の値ではありません。
- `value={keyword}` は「**入力欄に表示する中身を、状態 `keyword` にする**」指定。状態と入力欄の表示をひもづけます。
- `onChangeText={(text) => setKeyword(text)}` は「**入力が変わるたびに呼ばれる処理**」。変化後の文字が `text` に入って届くので、それを `setKeyword(text)` で状態に保存します。
- この2つがセットになることで「打つ→状態が変わる→その状態が入力欄に映る」という流れが完成します。

> **用語: 制御コンポーネント（controlled component）** とは、「入力欄の中身を state が管理している」状態のこと。`value` と `onChangeText` をセットで使うのがその基本形です。

解説3: 入力をリアルタイムで映す `{keyword}`

コピー

```
{/* 入力した文字がリアルタイムで表示される */}
<Text>入力中: {keyword}</Text>
```

- `<Text>` の中の `{keyword}` は、「**状態 `keyword` の中身を、ここに表示する**」という意味です。`{ }` で囲むとTypeScriptの値を埋め込めます。
- `keyword` は入力のたびに更新されるので、ここの表示も**打つそばからリアルタイムで変わります**。
- これは「状態が変わると画面が自動で更新される」というReactの仕組みを実感できる、わかりやすい例です。

> **用語: `{ }`（中カッコ）** は、JSXの中に「TypeScriptの値（変数や計算結果）」を埋め込むための記号。`{keyword}` で状態の中身を画面に差し込んでいます。

3.2 `Pressable` — タップを受け取る

`Pressable`（プレッサブル＝押せるもの）は、あらゆる部品を「押せるボタン」にできる万能の部品です。

> **▼ このコードがやること（先に日本語で）:** 「保存する」と書かれた青いボタンを作り、**押されたらメッセージを記録（ログ出力）する**例です。`<Pressable>` で中身を囲み、`onPress` に「押されたときにやってほしい処理」を渡すのが基本の形です。ボタンの中の文字もやはり `<Text>` で囲む点に注意してください。見た目の指定はコード内コメントで説明しています。

コピー

```
import { Pressable, Text, StyleSheet } from "react-native";

export default function SaveButton() {
  return (
    // onPress : 押されたときに実行する処理を渡す
    <Pressable style={styles.button} onPress={() => console.log("保存ボタンが押された")}>
      {/* ボタンの中身も<Text>で囲む */}
      <Text style={styles.buttonText}>保存する</Text>
    </Pressable>
  );
}

const styles = StyleSheet.create({
  button: {
    // 背景色（青）
    backgroundColor: "#1e40af",
    // 上下の内側余白
    paddingVertical: 13,
    // 角丸
    borderRadius: 10,
    // 中の文字を横中央に
    alignItems: "center",
  },
  buttonText: {
    // 文字色（白）
    color: "#fff",
    fontWeight: "bold",
    fontSize: 15,
  },
});
```

> **`Button` ではなく `Pressable` を使う理由:** React Nativeには標準の `<Button>` もありますが、見た目のカスタマイズがほとんどできません。デザインを自由にしたい実際のアプリでは `Pressable` が定番です。本書も以降は `Pressable` を使います。

▼ コードを1つずつ分解して解説

「押せるボタンを作り、押されたら処理する」部分を、1つずつ見ていきましょう。

解説1: 中身を囲んで押せるようにする `<Pressable>`

コピー

```
<Pressable style={styles.button} onPress={() => console.log("保存ボタンが押された")}>
  {/* ボタンの中身も<Text>で囲む */}
  <Text style={styles.buttonText}>保存する</Text>
</Pressable>
```

- `<Pressable>`（プレッサブル＝押せるもの）は、「**中に入れた部品を、タップできるボタンに変える**」万能の部品です。
- `style={styles.button}` でボタン自体の見た目（背景色や角丸など）を指定しています。
- ボタンの**中の文字も必ず** `<Text>` **で囲む**点に注意してください。`<Pressable>` の中に文字を直接書くとエラーになります。
- `<Pressable>` と `</Pressable>` で囲んだ範囲全体が、押せる領域になります。

> **用語: `<Pressable>`** はWebの `<button>` にあたる部品ですが、「中に何を入れてもボタンにできる」点が特徴。文字でも画像でも箱でも、囲めば押せるようになります。

解説2: 押されたときの処理 `onPress`

コピー

```
onPress={() => console.log("保存ボタンが押された")}
```

- `onPress`（オン・プレス＝押されたら）は、「**ボタンが押された瞬間に実行してほしい処理**」を渡す指定です。Webの `onClick` にあたります。
- `() => console.log("...")` は「**引数なしで、`console.log(...)` を実行するだけの関数**」を表すアロー関数です。この関数そのものを `onPress` に渡しています。
- `console.log(...)`（コンソール・ログ）は、開発者ツールの画面に文字を表示する命令で、動作確認によく使います。
- 実際のアプリでは、この中に「データを保存する処理」などを書きます。

> **用語: `() => { }`（アロー関数）** は「処理をひとかたまりにして渡す」ための短い関数の書き方。`onPress` には、すぐ実行するのではなく「押されたときに実行する関数」を渡すのがポイントです。

---

4. リストを表示する — `ScrollView` と `FlatList`

スマホは画面が小さいので、たくさんの本を表示するには**スクロール**が必須です。方法は2つあります。

4.1 `ScrollView` — 少数の要素向け

> **▼ このコードがやること（先に日本語で）**：いくつかの文字を縦に並べ、画面に収まりきらなければ**指でスクロールして見られる箱**を作ります。`<ScrollView>` で中身を囲むだけで、はみ出した分をスクロール表示できるようになります。手軽ですが「中身を全部いっぺんに表示する」ため、項目が少ないとき向けだと覚えておきましょう。

コピー

```
import { ScrollView, Text } from "react-native";

export default function SimpleList() {
  return (
    // ScrollView : 中身が画面を超えたらスクロールできる箱
    <ScrollView>
      <Text>本A</Text>
      <Text>本B</Text>
      <Text>本C</Text>
      {/* ...たくさん並べてもスクロールできる */}
    </ScrollView>
  );
}
```

> **`ScrollView` の注意点**：`ScrollView` は中身を**全部いっぺんに描画**します。本が10冊程度なら問題ありませんが、何百件もあるとアプリが重くなります。大量データには次の `FlatList` を使います。

▼ コードを1つずつ分解して解説

「囲むだけでスクロールできる箱」を、1つずつ見ていきましょう。

解説1: 中身を囲む `<ScrollView>`

コピー

```
<ScrollView>
  <Text>本A</Text>
  <Text>本B</Text>
  <Text>本C</Text>
  {/* ...たくさん並べてもスクロールできる */}
</ScrollView>
```

- `<ScrollView>`（スクロール・ビュー）は、「**中身が画面に収まりきらなければ、指でスクロールして見られる箱**」です。
- `<View>` と同じように中に部品を並べるだけで、はみ出した分を自動でスクロール表示できるようになります。特別な設定は不要です。

- ここでは `<Text>` を3つ並べていますが、もっとたくさん並べても、画面を超えた分はスクロールで見られます。
- 既定では**縦方向**のスクロールになります（横スクロールにしたいときは別途指定が必要）。

> **用語: `<ScrollView>`** は「スクロールできるView」。手軽ですが、中身を**全部いっぺんに作る**ため、項目が少ないとき向けです。大量データには次の `FlatList` を使います。

4.2 `FlatList` — 大量データ向け（本書の主役）

`FlatList`（フラットリスト）は「**画面に見えている分だけ描画**し、スクロールに応じて順次描画する」賢いリストです。本書の書籍一覧はこれで作ります。

> **▼ このコードがやること（先に日本語で）**：3冊の本のデータ（配列）を渡して、**1冊ずつカード状に並べたリスト**を作ります。`FlatList` には「表示したいデータ（ `data` ）」「1件分をどう表示するか（ `renderItem` ）」「各件を区別する目印（ `keyExtractor` ）」の3点を教えるのが基本です。データの配列を渡せば自動で繰り返し表示してくれる、という仕組みをつかんでください。各プロパティの意味はコード内コメントにあります。

コピー

```
import { FlatList, View, Text, StyleSheet } from "react-native";

// 表示する本のデータ（後の章ではSupabaseから取得するが、ここでは仮データ）
const books = [
  { id: "1", title: "リーダブルコード", author: "Dustin Boswell" },
  { id: "2", title: "達人プログラマー", author: "David Thomas" },
  { id: "3", title: "TypeScript入門", author: "鈴木 僚太" },
];

export default function BookList() {
  return (
    <FlatList
      // data : 表示するデータの配列を渡す
      data={books}
      // keyExtractor : 各要素を区別する一意のキーを返す（idを使う）
      keyExtractor={(item) => item.id}
      // renderItem : 1件分の見た目を返す関数。itemに1冊分が入る
      renderItem={({ item }) => (
        <View style={styles.card}>
          {/* その本のタイトル */}
          <Text style={styles.title}>{item.title}</Text>
          {/* その本の著者 */}
          <Text style={styles.author}>{item.author}</Text>
        </View>
      )}
    />
  );
}

const styles = StyleSheet.create({
  card: { padding: 14, borderBottomWidth: 1, borderBottomColor: "#e2e8f0" },
  title: { fontSize: 15, fontWeight: "bold" },
  author: { fontSize: 12, color: "#64748b", marginTop: 3 },
});
```

`FlatList` の3つの必須プロパティ: - `data` … 表示したいデータの配列。 - `renderItem` … 「1件分をどう表示するか」を返す関数。`{ item }` で1件ずつ受け取る。 - `keyExtractor` … 各要素に固有の「目印（key）」を付ける関数。Reactが要素を効率よく管理するために必要。本のidを使うのが定番です。

第3章で `map` を使ってカードを並べる話をしましたが、大量データでは `map` + `ScrollView` より `FlatList` の方が高速で、スマホアプリの定番です。

▼ コードを1つずつ分解して解説

`FlatList` に教える3つの基本情報を、1つずつ見ていきましょう。

解説1: 表示するデータの配列 `data`

コピー

```
const books = [
  { id: "1", title: "リーダブルコード", author: "Dustin Boswell" },
  { id: "2", title: "達人プログラマー", author: "David Thomas" },
  { id: "3", title: "TypeScript入門", author: "鈴木 僚太" },
];
```

コピー

```
// data : 表示するデータの配列を渡す
data={books}
```

- `books` は「本のデータの配列」です。`[ ]` の中に、本1冊分を `{ }`（オブジェクト）で表したものを並べています。各本は `id`・`title`・`author` の3項目を持ちます。
- `data={books}` は「**この配列を順番に表示してね**」と `FlatList` に渡す指定です。
- `FlatList` は渡された配列を見て、**1件ずつ自動で繰り返し表示**してくれます。自分で `map` を書く必要はありません。
- ここでは仮のデータですが、後の章ではこの部分をデータベース（Supabase）から取得した配列に差し替えます。

> **用語: 配列（はいれつ）** とは、複数のデータを順番に並べて1つにまとめたもの。`[ ]` で囲み、各要素を `,`（カンマ）で区切ります。`FlatList` はこの配列の中身を1件ずつ表示します。

解説2: 各要素を区別する目印 `keyExtractor`

コピー

```
// keyExtractor : 各要素を区別する一意のキーを返す（idを使う）
keyExtractor={(item) => item.id}
```

- `keyExtractor`（キー・エクストラクター＝キーを取り出すもの）は、「**各データに付ける固有の目印（key）を返す関数**」です。
- `(item) => item.id` は「1冊分（`item`）を受け取って、その `id` を目印として返す」という意味のアロー関数です。
- Reactはこの目印を使って「どの要素がどれか」を見分け、**並び替えや追加・削除を効率よく**処理します。
- 目印は配列の中で**重複しない値**である必要があるため、本の `id`（一意な番号）を使うのが定番です。

> **用語: key（キー）** とは、リストの各要素に付ける「見分けるための名札」。Reactが要素を効率よく管理するために必要で、重複しない値（idなど）を使います。

解説3: 1件分の見た目を返す `renderItem`

コピー

```
// renderItem : 1件分の見た目を返す関数。itemに1冊分が入る
renderItem={({ item }) => (
  <View style={styles.card}>
    {/* その本のタイトル */}
    <Text style={styles.title}>{item.title}</Text>
    {/* その本の著者 */}
    <Text style={styles.author}>{item.author}</Text>
  </View>
)}
```

- `renderItem`（レンダー・アイテム＝1件を描く）は、「**配列の1件を、どんな見た目で表示するか**」を返す関数です。`FlatList` がデータ1件ごとにこの関数を呼びます。
- `({ item }) => ( ... )` の `item` には**1冊分のデータ**が入ります（`{ item }` は分割代入で、届いた情報の中から `item` だけ取り出す書き方）。
- `{item.title}` と `{item.author}` で、その本のタイトルと著者を取り出して表示しています。
- 結果として、3冊それぞれがこの見た目（カード）で繰り返し表示されます。

> **用語: `renderItem`** は「1件をどう描くか」のテンプレート（ひな型）。`FlatList` はこのひな型に、配列の各データを1つずつ当てはめて画面を作ります。

5. Expo Router — 画面の切り替え（ナビゲーション）

アプリには複数の画面があり、それらを行き来します（一覧→詳細→編集など）。この「画面遷移（ナビゲーション）」を担うのが **Expo Router（エクスポ・ルーター）** です。

5.1 ファイルを置くだけで画面ができる

Expo Routerの最大の特徴は「**`app/` フォルダにファイルを置くと、それがそのまま1つの画面（ルート）になる**」ことです。Web版チュートリアルのNext.jsと同じ「ファイルベースルーティング」の考え方です。

コピー

```
app/
├── index.tsx          → アプリ起動時の最初の画面（URLでいう「/」）
├── about.tsx          → /about でアクセスする画面
└── books/
    └── [id].tsx       → /books/1, /books/2 ... のような「動的な」画面
```

> 「**ルーティング（routing）」とは？**「どのURL（道筋）のとき、どの画面を表示するか」を決める仕組みのこと。「ルート（route）＝道筋」を割り当てるので routing です。Expo Routerでは、ファイルの場所がそのままルートになります。
>
> **`[id].tsx` の角カッコは何？** ファイル名を `[id].tsx` のように角カッコで囲むと、「**どんな値でも受け取れる動的な画面**」になります。`/books/1` でも `/books/99` でも、この1つのファイルが対応し、`id` の部分（1や99）を受け取れます。本の詳細・編集画面に使います（第6・8章）。

5.2 画面を移動する — `Link` と `useRouter`

別の画面へ移動する方法は2つあります。

> **▼ このコードがやること（先に日本語で）**：押すと別の画面（ここでは `/about` ）へ移動する**リンク部品**を置く例です。`<Link>` で文字などを囲み、`href` に「移動先のパス（場所）」を書くだけで、タップで画面が切り替わります。Webのリンクと同じ感覚で、「画面に置いておく移動ボタン」を作りたいときに使います。

コピー

```
// 方法1: <Link> – 押すと移動するリンク部品
import { Link } from "expo-router";
import { Text } from "react-native";

function Example1() {
  return (
    {/* href : 移動先のパス。押すと/about画面へ */}
    <Link href="/about">
      <Text>Aboutページへ</Text>
    </Link>
  );
}
```

▼ コードを1つずつ分解して解説

「画面に置く移動リンク」を、1つずつ見ていきましょう。

解説1: Link部品を借りてくる `import`

コピー

```
import { Link } from "expo-router";
import { Text } from "react-native";
```

- `Link`（リンク）は `expo-router` から借りてきます。これが「**押すと別の画面へ移動する部品**」です。

- 移動の仕組みは `expo-router`（Expo Router）が担当するので、`react-native` ではなく `expo-router` から借りる点に注意します。
- 中に表示する文字のために `Text` も `react-native` から借りています。

> **用語: Expo Router** とは、Expoアプリの「画面の切り替え（ナビゲーション）」を担う仕組み。`app/` フォルダに置いたファイルが画面になり、その間の移動を `Link` などが担当します。

解説2: 移動先を指定する `href`

コピー

```
{/* href : 移動先のパス。押すと/about画面へ */}
<Link href="/about">
  <Text>Aboutページへ</Text>
</Link>
```

- `<Link>` で囲んだ中身（ここでは「Aboutページへ」という文字）が、**タップできるリンク**になります。
- `href`（エイチレフ＝参照先）には「**移動先のパス（場所）**」を書きます。`"/about"` は「`/about` という画面へ」という意味です。
- Webのリンク（`<a href="...">`）とほぼ同じ感覚で使えます。押すとその画面に切り替わります。
- 「画面にずっと置いておく移動ボタン」を作りたいときに向いています。

> **用語: パス（path）** とは「どの画面か」を表す住所のような文字列。Expo Routerでは `app/about.tsx` というファイルが `/about` というパスに対応します。

> **▼ このコードがやること（先に日本語で）**：ボタンを押したときに、**コードの中から別の画面へ移動させる**例です。`useRouter` で「移動を操作する道具」を受け取り、`router.push("/移動先")` を呼ぶとその画面へ進みます。「処理が終わってから画面を切り替えたい」ような、流れの途中で移動したい場面で使う方法です。

コピー

```
// 方法2: useRouter – コードの中から移動する（ボタンを押した後など）
import { useRouter } from "expo-router";
import { Pressable, Text } from "react-native";

function Example2() {
  // useRouter : 画面移動を操作する道具を取得
  const router = useRouter();

  return (
    {/* router.push("パス") でその画面へ移動 */}
    <Pressable onPress={() => router.push("/books/1")}>
      <Text>1冊目の詳細へ</Text>
    </Pressable>
  );
}
```

▼ コードを1つずつ分解して解説

「コードの中から画面を移動させる」部分を、1つずつ見ていきましょう。

解説1: 移動の道具を受け取る `useRouter`

コピー

```
// useRouter : 画面移動を操作する道具を取得
const router = useRouter();
```

- `useRouter`（ユーズ・ルーター）は、「**画面移動を操作するための道具を受け取る**」関数です。`expo-router` から借りて使います。
- 受け取った道具を `router` という変数に入れておき、後でこれを使って移動させます。
- `<Link>` のように画面に置くのではなく、**コードの流れの中で「今移動して」と命令したいとき**に使います。

> **用語: use〜 で始まる関数（フック）** は、Reactの便利機能を呼び出すための道具。`useState` が状態を、`useRouter` が画面移動の道具を用意してくれます。

解説2: 指定の画面へ進む `router.push`

コピー

```
{/* router.push("パス") でその画面へ移動 */}
<Pressable onPress={() => router.push("/books/1")}>
  <Text>1冊目の詳細へ</Text>
</Pressable>
```

- `onPress={() => router.push("/books/1")}` は「**押されたら `router.push("/books/1")` を実行する**」という指定です。
- `router.push("パス")` は「**そのパスの画面へ進む（新しい画面を上に重ねる）**」命令です。ここでは `/books/1`（1冊目の詳細画面）へ移動します。
- `Link` と違い、コードの中で呼ぶので「**保存処理が終わってから移動**」のように、流れの途中で移動させたいときに便利です。
- 1つ前の画面に戻りたいときは `router.back()` を使います。

> **用語: `router.push()`** は「画面を上に積み重ねる移動」。トランプのカードを重ねるイメージで、`router.back()` で上のカードをめくって前の画面に戻れます。

> **`Link` と `router.push` の使い分け:** - **`<Link>`** ...「押したら移動するリンク」を画面に置きたいとき。 - **`router.push()`** ...「保存ボタンを押して、処理が終わったら一覧画面へ戻す」のように、コードの流れの中で移動したいとき。
>
> `router.push("/path")` は新しい画面を上に重ねる移動、`router.back()` は1つ前の画面に戻る移動です。第6章以降で実際に使います。

5.3 `_layout.tsx` — 画面全体の枠組み

`app/` フォルダ内の `_layout.tsx`（アンダースコアで始まる特別なファイル）は、「全画面に共通する枠組み」を定義します。たとえば「画面上部のヘッダー」や「下部のタブバー」をここで設定します。

> **▼ このコードがやること（先に日本語で）**：すべての画面に共通する「枠組み」を決める、いちばん簡単な `_layout.tsx` の例です。`<Stack />` を返すことで、「画面をカードのように上へ積み重ねて切り替える」遷移方式を指定しています。今は中身が1行だけですが、ここに共通のヘッダーやタブバーを足していく土台になる、という位置づけを押さえてください。

コピー

```
import { Stack } from "expo-router";

// _layout.tsx : このフォルダ内の画面たちを、どう積み重ねる(Stack)かを定義する
export default function RootLayout() {
  // Stack : 画面を「カードを重ねるように」遷移させるナビゲーションの形
  return <Stack />;
}
```

> **「Stack（スタック）」ナビゲーションとは？** 画面を「トランプのカードを上に重ねていく」ように遷移させる方式です。一覧画面の上に詳細画面を重ね、「戻る」で上のカードをめくって一覧に戻る、というイメージ。スマホアプリで最も基本的な遷移方式です。第6章でこのStackと、下タブの「Tabs」ナビゲーションを実際に組み立てます。

▼ コードを1つずつ分解して解説

「全画面に共通する枠組み」を決める部分を、1つずつ見ていきましょう。

解説1: Stack部品を借りてくる `import`

コピー

```
import { Stack } from "expo-router";
```

- `Stack`（スタック＝積み重ね）は `expo-router` から借りてきます。これが「**画面をカードのように上へ積み重ねて切り替える**」遷移方式を表す部品です。
- 移動の仕組みはExpo Routerが担当するので、ここでも `expo-router` から借ります。

> **用語: `_layout.tsx`** は、先頭にアンダースコア `_` が付いた特別なファイル名。「画面そのもの」ではなく「**そのフォルダ内の画面たちに共通する枠組み**」を定義する、という意味を持ちます。

解説2: 遷移方式を指定する `<Stack />`

コピー

```
export default function RootLayout() {
  // Stack : 画面を「カードを重ねるように」遷移させるナビゲーションの形
  return <Stack />;
}
```

- `RootLayout` は「このフォルダの枠組み」を表すコンポーネントです。`export default` で公開し、Expo Routerがこれを枠組みとして使います。

- `return <Stack />;` は「**この枠組みではStack（カードを重ねる）方式で画面を切り替える**」という指定です。`<Stack />` は中身のない自己閉じタグで書きます。
- いまは1行だけですが、ここに共通のヘッダーや下部のタブバーを足していく**土台**になります。第6章でその中身を組み立てます。

> **用語: レイアウト（layout）** とは「複数の画面に共通する外枠」のこと。ヘッダーやタブバーなど、どの画面でも同じように出したい部分を `_layout.tsx` にまとめます。

---

6. 安全な表示領域 — `SafeAreaView`

最近のスマホは画面上部にノッチ（カメラ部分の切り欠き）や、下部にホームバーがあります。これらに文字が隠れないようにする部品が `SafeAreaView` です。

> **▼ このコードがやること（先に日本語で）**：文字が、画面上部のノッチ（カメラの切り欠き）や下部のホームバーに**重なって隠れないように表示する**例です。中身を `<SafeAreaView>` で囲むだけで、機種ごとの「避けるべき危険地帯」を自動でよけてくれます。普通の `<View>` の代わりに一番外側の箱として使う、という使い方を覚えておきましょう。

コピー

```
// この部品集から借りる
import { SafeAreaView } from "react-native-safe-area-context";
import { Text } from "react-native";

export default function Screen() {
  return (
    // SafeAreaView : ノッチやホームバーを避けた「安全な領域」に中身を表示する箱
    <SafeAreaView style={{ flex: 1 }}>
      <Text>この文字はノッチに隠れません</Text>
    </SafeAreaView>
  );
}
```

> **なぜ必要？** 普通の `<View>` だと、文字がカメラのノッチやステータスバーに重なって読めなくなることがあります。`SafeAreaView` で囲むと、機種ごとの「危険地帯」を自動で避けてくれます。Expoのテンプレートには最初から組み込まれていることが多いです。

▼ コードを1つずつ分解して解説

「安全な領域に表示する箱」を、1つずつ見ていきましょう。

解説1: SafeAreaView を借りてくる `import`

コピー

```
// この部品集から借りる
import { SafeAreaView } from "react-native-safe-area-context";
import { Text } from "react-native";
```

- `SafeAreaView`（セーフ・エリア・ビュー＝安全領域のView）は、`react-native-safe-area-context` という**別の部品集**から借ります。
- `View` / `Text` のように `react-native` 本体から借りるのではない点に注意します。借りる場所（`from "..."`）を間違えるとエラーになります。
- このライブラリはExpoのテンプレートに最初から入っていることが多く、すぐ使えます。

> **用語: ノッチ** とは画面上部のカメラ部分の切り欠き、**ホームバー** とは画面下部の操作バーのこと。これらに文字が重なると読めなくなるため、避ける必要があります。

解説2: 中身を安全領域に表示する `<SafeAreaView>`

コピー

```
<SafeAreaView style={{ flex: 1 }}>
  <Text>この文字はノッチに隠れません</Text>
</SafeAreaView>
```

- `<SafeAreaView>` で囲むだけで、中身が**ノッチやホームバーを避けた「安全な領域」に表示**されます。機種ごとの避けるべき範囲は自動で計算されます。
- 普通の `<View>` の代わりに、**一番外側の箱**として使うのが基本の使い方です。
- `style={{ flex: 1 }}` は「画面いっぱいに広がる」指定。安全領域の中で、中身を画面全体に広げています。

> **用語: `SafeAreaView`** は「安全領域（safe area）の中だけに表示するView」。機種ごとに違うノッチやホームバーの位置を自動でよけてくれるので、文字が隠れる心配がなくなります。

発展: アプリでは使っていない重要なReact Native機能

> ここまでで、本書のアプリを作るのに必要な部品はひととおり登場しました。このセクションでは、本書のアプリでは直接使っていないけれど、**実際のスマホアプリ作りでとてもよく使う重要な機能**を6つ紹介します。どれも「いつ使うか・なぜ必要か」から説明するので、必要になったときに「あ、あれが使える」と思い出せるようになっておきましょう。各トピックは独立した最小サンプルなので、気になるものから読んで構いません。

Image — 画像を表示する

> ▼ **このコードがやること（先に日本語で）**：アプリに**画像を2種類の方法で表示する**例です。1つはアプリに同梱した画像ファイル（ローカル画像）、もう1つはインターネット上にある画像（ネットワーク画像）です。どちらも `<Image>` という部品で表示しますが、画像の「指定のしかた」が少し違います。スマホでは画像の**大きさを自分で指定する**必要がある点も大事なポイントです。各指定の意味はコード内コメントにあります。

いつ使う・なぜ必要？ 本の表紙、プロフィール写真、アイコンなど、アプリに画像を出したい場面はとても多いです。Webの `<img>` は使えないので、React Native専用の `<Image>` を使います。

コピー

```
import { View, Image, StyleSheet } from "react-native";

export default function ImageExample() {
  return (
    <View>
      {/* 方法1: ローカル画像（アプリに同梱したファイル）は require(...) で指定 */}
      <Image
        // source : 表示する画像。requireでファイルを読み込む
        source={require("../assets/cover.png")}
        // 大きさは style で指定する
        style={styles.local}
      />

      {/* 方法2: ネットワーク画像（インターネット上の画像）は { uri: "URL" } で指定 */}
      <Image
        // uri : 画像のあるURL（住所）
        source={{ uri: "https://example.com/book.png" }}
        // ネット画像は大きさ指定が必須
        style={styles.remote}
      />
    </View>
  );
}

const styles = StyleSheet.create({
  // width:横幅 / height:高さ（数値で指定）
  local: { width: 100, height: 150 },
  remote: { width: 100, height: 150 },
});
```

> **画像の大きさは自分で指定する:** Webと違い、React Nativeでは（特にネットワーク画像で） `width` と `height` を `style` で指定しないと画像が表示されないことがあります。「画像を出すときは大きさもセットで指定する」と覚えておきましょう。

▼ コードを1つずつ分解して解説

「2種類の画像指定」を、1つずつ見ていきましょう。

解説1: 同梱ファイルを表示する `require(...)`

コピー

```
<Image
  // source : 表示する画像。requireでファイルを読み込む
  source={require("../assets/cover.png")}
  // 大きさは style で指定する
  style={styles.local}
/>
```

- `<Image>`（イメージ＝画像）は、Webの `<img>` にあたる「**画像を表示する部品**」です。`source`（ソース＝もと）に「どの画像を出すか」を渡します。
- `require("../assets/cover.png")` は「**アプリに一緒に入れておいた画像ファイルを読み込む**」書き方です。`"../assets/cover.png"` はそのファイルの置き場所（パス）です。
- このように同梱した画像（ローカル画像）は、`require(...)` でファイルそのものを指定します。文字列のパスをそのまま渡すのではない点に注意してください。
- `style` で `width`・`height`（大きさ）を指定します。指定しないと表示されなかったり、意図しない大きさになります。

> **用語: ローカル画像** とは、アプリの中に一緒に入れて配布する画像ファイルのこと。アイコンやロゴなど「最初から決まっている画像」に使います。`require(...)` でアプリの中のファイルを読み込みます。

解説2: ネット上の画像を表示する `{ uri: "URL" }`

コピー

```
<Image
  // uri : 画像のあるURL（住所）
  source={{ uri: "https://example.com/book.png" }}
  // ネット画像は大きさ指定が必須
  style={styles.remote}
/>
```

- ネットワーク画像（インターネット上にある画像）は、`source={{ uri: "URL" }}` の形で「**画像のあるURL（住所）**」を渡します。
- 外側の `{ }` は「JSXの中にTypeScriptを書く印」、内側の `{ uri: "..." }` は「`uri` という項目を持つオブジェクト」です（`<View style={{...}}>` と同じ二重カッコのしくみ）。
- ネットワーク画像は、読み込んでみるまで大きさが分からないため、`width` と `height` **の指定が特に重要**です。指定しないと表示されないことがあります。
- 本の表紙をデータベースのURLから出す、といった場面ではこちらの方法を使います。

> **用語: URI（ユーアールアイ）** は、ここでは画像のある場所を表す「URL（インターネット上の住所）」とほぼ同じ意味。`{ uri: "https://..." }` で「この住所の画像を取ってきて表示して」と指定します。

Modal — ポップアップを表示する

> **▼ このコードがやること（先に日本語で）：** ボタンを押すと、画面の上に**小さなダイアログ（ポップアップ）が重なって出る**例です。`<Modal>` という部品で囲んだ中身が、ふだんは隠れていて、必要なときだけ前面に出ます。出す・隠すの切り替えは `useState`（状態）で管理します。「確認メッセージを出す」「設定パネルをかぶせる」といった場面の基本形です。各指定の意味はコード内コメントにあります。

いつ使う・なぜ必要？「本当に削除しますか?」の確認や、入力フォームを一時的にかぶせて表示したいときに使います。別の画面に移動せず、**今の画面の上に重ねて出す**のがModalの役割です。

コピー

```
import { useState } from "react";
import { View, Text, Pressable, Modal, StyleSheet } from "react-native";

export default function ModalExample() {
  // ポップアップを出すかどうかの状態（最初は隠す=false）
  const [visible, setVisible] = useState(false);

  return (
    <View style={styles.center}>
      {/* このボタンでポップアップを出す（visibleをtrueに） */}
      <Pressable onPress={() => setVisible(true)}>
        <Text>ポップアップを開く</Text>
      </Pressable>

      <Modal
        // visible : trueなら表示、falseなら非表示
        visible={visible}
        // transparent : 背景を透けさせて、後ろの画面をうっすら見せる
        transparent={true}
        // animationType : 出入りの動き（fade=じわっと）
        animationType="fade"
        // Android戻るボタンで閉じたときの処理
        onRequestClose={() => setVisible(false)}
      >
        {/* 画面全体を薄暗くする背景 */}
        <View style={styles.overlay}>
          <View style={styles.box}>
            <Text style={styles.message}>これはポップアップです</Text>
            {/* このボタンで閉じる（visibleをfalseに） */}
            <Pressable onPress={() => setVisible(false)}>
              <Text style={styles.close}>閉じる</Text>
            </Pressable>
          </View>
        </View>
      </Modal>
    </View>
  );
}

const styles = StyleSheet.create({
  center: { flex: 1, justifyContent: "center", alignItems: "center" },
  // 後ろを薄暗くする全画面の背景
  overlay: {
    flex: 1,
    justifyContent: "center",
    alignItems: "center",
    // 黒の半透明（0.4=40%の濃さ）
    backgroundColor: "rgba(0,0,0,0.4)",
```

```
  },
  // 白いカード
  box: { backgroundColor: "#fff", padding: 24, borderRadius: 12 },
  message: { fontSize: 16, marginBottom: 16 },
  close: { color: "#1e40af", fontWeight: "bold" },
});
```

▼ コードを1つずつ分解して解説

「出す・隠すを切り替えられるポップアップ」を、1つずつ見ていきましょう。

解説1: 出すか隠すかを覚えておく状態

コピー

```
// ポップアップを出すかどうかの状態（最初は隠す=false）
const [visible, setVisible] = useState(false);
```

- `useState(false)` で「**ポップアップを今出しているか**」を覚えておく状態を作ります。`true` なら出す、`false` なら隠す、という意味です。
- 最初は `false`（隠す）にしているので、画面を開いた直後はポップアップは出ていません。
- `setVisible(true)` で出し、`setVisible(false)` で隠します。この状態1つで、ポップアップの表示・非表示を切り替えます。

> **用語: 真偽値（しんぎち / boolean）** とは「はい（`true`）か いいえ（`false`）」の2つだけを表す値。「出す/出さない」「オン/オフ」のような2択の状態を覚えるのにぴったりです。

解説2: 重ねて表示する `<Modal>`

コピー

```
<Modal
  // visible : trueなら表示、falseなら非表示
  visible={visible}
  // transparent : 背景を透けさせて、後ろの画面をうっすら見せる
  transparent={true}
  // animationType : 出入りの動き（fade=じわっと）
  animationType="fade"
  // Android戻るボタンで閉じたときの処理
  onRequestClose={() => setVisible(false)}
>
```

- `<Modal>`（モーダル）は、「**今の画面の上に、別の中身を重ねて表示する**」部品です。

- `visible={visible}` で「出すか隠すか」を状態とひもづけます。状態が `true` のときだけ中身が前面に出ます。
- `transparent={true}` は「**背景を透明にして、後ろの画面をうっすら見せる**」指定。これと薄暗い背景（後述の `overlay`）を組み合わせると、よくある「後ろが暗くなってポップアップが浮かぶ」見た目になります。
- `animationType="fade"` は出入りの動きで、`"fade"` はじわっと現れる動きです（他に `"slide"` などがあります）。
- `onRequestClose` は、Androidの「戻る」ボタンで閉じようとしたときに呼ばれる処理です。ここで `setVisible(false)` して閉じます。

> **用語: モーダル（modal）** とは「いったんそれを操作（または閉じる）しないと先に進めない、前面に重なる表示」のこと。確認ダイアログや設定パネルなど、「今これに注目してほしい」ものに使います。

解説3: 後ろを薄暗くする背景とカード

コピー

```
<View style={styles.overlay}>
  <View style={styles.box}>
    <Text style={styles.message}>これはポップアップです</Text>
    <Pressable onPress={() => setVisible(false)}>
      <Text style={styles.close}>閉じる</Text>
    </Pressable>
  </View>
</View>
```

- `styles.overlay` の `backgroundColor: "rgba(0,0,0,0.4)"` は「**黒の半透明**」で、画面全体をうっすら暗くする背景です。これで後ろの画面が暗くなり、ポップアップが浮かんで見えます。
- その中の `styles.box`（白いカード）が、実際のポップアップの中身です。`justifyContent`・`alignItems` の中央寄せで、画面の真ん中に置いています。
- 「閉じる」を押すと `setVisible(false)` が呼ばれ、状態が `false` になってポップアップが消えます。
- このように「薄暗い背景＋中央の白いカード」を自分で組むのが、見た目を整えたModalの定番パターンです。

> **用語: `rgba(0,0,0,0.4)`** は色の指定方法の1つで、`rgba(赤, 緑, 青, 透明度)` の形。最後の `0.4` が透明度で、`0` =完全に透明、`1` =完全に不透明。`0.4` は「40%の濃さの黒」を意味します。

Platform — iOS と Android で出し分ける

> **▼ このコードがやること（先に日本語で）：今アプリが動いているのがiPhone（iOS）かAndroidかを判定して、表示や設定を変える**例です。`Platform.OS` で「今どっちの端末か」が分かり、`Platform.select` で「iOSならコレ、Androidならコレ」と値を切り替えられます。OSによって少しだけ見た目や余白を変えたいときに使います。各指定の意味はコード内コメントにあります。

いつ使う・なぜ必要？ iOSとAndroidは、ちょうどよい余白やフォント、操作感が少しずつ違います。「iOSだけ上に余白を足す」「OSごとに違うフォントを使う」など、**端末によって細かく調整したい**ときに使います。1つのコードで両方のアプリを作るReact Nativeならではの機能です。

コピー

```
import { View, Text, Platform, StyleSheet } from "react-native";

export default function PlatformExample() {
  return (
    <View style={styles.box}>
      {/* Platform.OS : 今動いているOSの名前。"ios" か "android" が入る */}
      <Text>今のOS: {Platform.OS}</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  box: {
    // Platform.select : OSごとに違う値を選ぶ。iosのときは上余白を多めに、androidは少なめに
    paddingTop: Platform.select({ ios: 50, android: 20 }),
    backgroundColor: "#f1f5f9",
  },
});
```

▼ コードを1つずつ分解して解説

「OSを見分けて出し分ける」2つの書き方を、1つずつ見ていきましょう。

解説1: 今のOSを調べる `Platform.OS`

コピー

```
<Text>今のOS: {Platform.OS}</Text>
```

- `Platform`（プラットフォーム）は、React Native本体から借りる「**今の端末の情報を教えてくれる道具**」です。
- `Platform.OS` には、今アプリが動いている端末の種類が入っています。iPhoneなら `"ios"`、Androidなら `"android"` という文字が入ります。
- `{Platform.OS}` のように `{ }` で囲んでJSXに埋め込むと、その値（`"ios"` など）を画面に表示できます。

- これを `if` などと組み合わせて「iOSのときだけこうする」といった分岐に使えます。

> **用語: OS（オーエス）** とは、スマホを動かしている基本ソフトのこと。iPhoneは「iOS」、多くの他社スマホは「Android」というOSで動いています。React Nativeは1つのコードで両方のOS向けアプリを作れます。

解説2: OSごとに値を選ぶ `Platform.select`

コピー

```
paddingTop: Platform.select({ ios: 50, android: 20 }),
```

- `Platform.select({ ... })` は「**OSごとに違う値を選んで返す**」道具です。`{ ios: ..., android: ... }` のように、OSごとの値を書いておきます。
- ここでは「iOSなら `paddingTop`（上の内側余白）を `50`、Androidなら `20`」という意味になります。実際に動いているOSに合った値が自動で選ばれます。
- `Platform.OS` を `if` で分岐して書くこともできますが、`Platform.select` を使うと**1行ですっきり**書けて便利です。
- スタイルの中で「OSごとにこの値だけ変えたい」というときの定番の書き方です。

> **用語: `Platform.select`** は「OSごとに用意した値の中から、今のOSに合うものを選ぶ」しくみ。`{ ios: ..., android: ... }` という形で、それぞれのOS向けの値をまとめて渡します。

Dimensions と useWindowDimensions — 画面の幅・高さを知る

> **▼ このコードがやること（先に日本語で）：今使っている端末の画面の幅や高さ（ピクセル数）を取得して、表示に使う**例です。`useWindowDimensions` という道具を使うと、画面の幅・高さが分かり、しかも**画面を回転させたときなどに自動で更新**されます。「画面の幅に合わせて部品の大きさを変えたい」ときに役立ちます。各指定の意味はコード内コメントにあります。

いつ使う・なぜ必要？ スマホは機種によって画面の大きさがバラバラです。「画面の幅いっぱいの半分の大きさにしたい」「横向きにしたら並べ方を変えたい」など、**画面サイズに合わせて見た目を調整したい**ときに、今の画面サイズを知る必要があります。

コピー

```
import { useWindowDimensions, View, Text, StyleSheet } from "react-native";

export default function DimensionsExample() {
  // useWindowDimensions : 今の画面の幅・高さを取得する道具。回転などで変わると自動で更新される
  const { width, height } = useWindowDimensions();

  return (
    <View>
      {/* Math.round : 小数を四捨五入して整数に */}
      <Text>画面の幅: {Math.round(width)}</Text>
      <Text>画面の高さ: {Math.round(height)}</Text>

      {/* 画面幅の半分の大きさの四角を表示する */}
      <View style={[styles.box, { width: width / 2 }]} />
    </View>
  );
}

const styles = StyleSheet.create({
  box: { height: 60, backgroundColor: "#1e40af", marginTop: 12 },
});
```

> **Dimensions との違い:** `import { Dimensions } from "react-native"` を使い `Dimensions.get("window")` でも画面サイズを取れますが、こちらは**取得した瞬間の値**なので、画面を回転させても自動では変わりません。画面の回転にも対応したいときは、自動更新される **`useWindowDimensions`**（フック）のほうが安全で、今はこちらが推奨です。

▼ コードを1つずつ分解して解説

「画面サイズを取得して使う」部分を、1つずつ見ていきましょう。

解説1: 画面の幅・高さを受け取る `useWindowDimensions`

コピー

```
const { width, height } = useWindowDimensions();
```

- `useWindowDimensions`（ユーズ・ウィンドウ・ディメンションズ）は、「**今の画面の幅と高さを受け取る**」道具（フック）です。`use〜` で始まるReactの便利機能の仲間です。
- `const { width, height } = ...` は、受け取った情報の中から `width`（幅）と `height`（高さ）の2つを取り出す書き方（分割代入）です。
- このフックの便利な点は、**画面を回転させたりサイズが変わったりすると、自動で新しい値に更新される**ことです。そのたびに画面も作り直されます。
- 取り出した `width`・`height` は、ピクセル数の数値なので、そのまま計算（`width / 2` など）に使えます。

> **用語: ピクセル** とは画面を構成する点の単位。`width` が `390` なら「画面の横幅は390ぶん」という意味で、これを基準に部品の大きさを計算できます。

解説2: 取得した幅を使って大きさを決める

コピー

```
<View style={[styles.box, { width: width / 2 }]} />
```

- `style={[ ... ]}` のように `[ ]`（配列）で囲むと、「**複数のスタイルを重ねて適用**」できます。ここでは `styles.box` と `{ width: width / 2 }` の2つを重ねています。
- `{ width: width / 2 }` は「幅を、画面の幅の半分にする」という指定です。`width`（画面幅）を取得しているので、それを `2` で割っています。
- こうすると、どんな機種でも「画面のちょうど半分の幅の四角」になります。機種ごとに数値を決め打ちしなくてよいのが利点です。
- `Math.round(width)` の `Math.round` は「小数を四捨五入して整数にする」命令で、表示を見やすくするために使っています。

> **用語: スタイルの配列** `[styleA, styleB]` は「複数のスタイルを順番に重ねる」書き方。後のものが優先されるので、共通のスタイルに「その場だけの追加指定」を足したいときに便利です。

AsyncStorage — 端末内にデータを保存する

> **▼ このコードがやること（先に日本語で）：入力した文字を端末の中に保存しておき、次にアプリを開いたときに読み出して表示する**例です。`AsyncStorage`（アシンク・ストレージ）という道具を使うと、アプリを閉じても消えないデータを端末内に残せます。「名前を保存→アプリ再起動→さっきの名前が出る」という、設定や下書きの保存によくある形です。各処理の意味はコード内コメントにあります。

いつ使う・なぜ必要？ `useState` で覚えた値は、アプリを閉じると消えてしまいます。「ダークモードのオン/オフ」「最後に入力した下書き」「ログイン状態」など、**アプリを閉じても残しておきたい小さなデータ**を端末内に保存するのに使います。これは追加の部品なので、`@react-native-async-storage/async-storage` をインストールして使います。

コピー

```
import { useState, useEffect } from "react";
import AsyncStorage from "@react-native-async-storage/async-storage";
import { View, Text, TextInput, Pressable, StyleSheet } from "react-native";

// 保存するときの「名札（キー）」。これで後から取り出す
const STORAGE_KEY = "user_name";

export default function StorageExample() {
  const [name, setName] = useState("");

  // アプリを開いたとき、保存済みの値を読み出す
  useEffect(() => {
    const load = async () => {
      // getItem : キーで値を読み出す
      const saved = await AsyncStorage.getItem(STORAGE_KEY);
      // 保存があれば状態に反映
      if (saved !== null) setName(saved);
    };
    load();
  // [] : 最初の1回だけ実行
  }, []);

  // 「保存」ボタンで、今の入力を端末に書き込む
  const save = async () => {
    // setItem : キーと値を渡して保存する
    await AsyncStorage.setItem(STORAGE_KEY, name);
  };

  return (
    <View style={styles.box}>
      <TextInput
        style={styles.input}
        placeholder="名前を入力"
        value={name}
        onChangeText={setName}
      />
      <Pressable onPress={save}>
        <Text style={styles.save}>保存する</Text>
      </Pressable>
    </View>
  );
}

const styles = StyleSheet.create({
  box: { padding: 16, gap: 12 },
  input: { borderWidth: 1, borderColor: "#e2e8f0", borderRadius: 8, padding: 10 },
```

```
  save: { color: "#1e40af", fontWeight: "bold" },
});
```

> **保存できるのは「文字」だけ:** AsyncStorageに保存できるのは文字列（文字）だけです。数値やオブジェクト・配列を保存したいときは、いったん `JSON.stringify(...)` で文字に変換して保存し、読み出すときに `JSON.parse(...)` で元に戻します。

▼ コードを1つずつ分解して解説

「保存して、次回読み出す」流れを、1つずつ見ていきましょう。

解説1: 保存する場所の名札 `STORAGE_KEY`

コピー

```
// 保存するときの「名札（キー）」。これで後から取り出す
const STORAGE_KEY = "user_name";
```

- `AsyncStorage` は、データを「**名札（キー）と中身（値）のセット**」で保存します。`STORAGE_KEY` はその名札にあたる文字です。
- 保存するときも読み出すときも**同じ名札**を使うことで、「さっき保存したあのデータ」を正しく取り出せます。名札がずれると別のデータになってしまいます。
- ここでは `"user_name"` という名札にしました。複数のデータを保存するときは、それぞれ別の名札を付けます。

> **用語: キー（key）と値（value）** とは「名札と中身」のペアのこと。ロッカーに例えると、キーが「ロッカー番号」、値が「中に入れる荷物」。番号（キー）を覚えておけば、後で荷物（値）を取り出せます。

解説2: 値を読み出す `getItem`

コピー

```
useEffect(() => {
  const load = async () => {
    // getItem : キーで値を読み出す
    const saved = await AsyncStorage.getItem(STORAGE_KEY);
    // 保存があれば状態に反映
    if (saved !== null) setName(saved);
  };
  load();
// [] : 最初の1回だけ実行
}, []);
```

- `useEffect`（ユーズ・エフェクト）は「**画面が表示されたタイミングで何かする**」ための道具です。最後の `[]` は「最初の1回だけ実行する」という意味です。
- `AsyncStorage.getItem(STORAGE_KEY)` は「**その名札で保存された値を読み出す**」命令です。読み出しには少し時間がかかるため、`await`（待つ）を付けて結果を待ちます。
- `await` を使うには関数に `async`（非同期）を付ける必要があるため、`async () => { ... }` の形にしています。
- まだ何も保存されていないと `null`（からっぽ）が返るので、`if (saved !== null)` で「保存があれば」だけ状態に反映しています。

> **用語: `async` / `await`** は「**時間のかかる処理の終わりを待つ**」ための書き方。`await` を付けると「この処理が終わるまで待ってから次へ進む」という意味になり、読み込みや保存のように一瞬では終わらない処理で使います。

解説3: 値を保存する `setItem`

コピー

```
const save = async () => {
  // setItem : キーと値を渡して保存する
  await AsyncStorage.setItem(STORAGE_KEY, name);
};
```

- `AsyncStorage.setItem(名札, 値)` は「**名札を付けて値を端末に保存する**」命令です。ここでは `STORAGE_KEY`（名札）と `name`（今の入力）を渡して保存しています。
- 保存にも少し時間がかかるので、`await` を付けて待ちます。そのため `save` も `async` 関数にしています。
- このボタンを押して保存しておけば、アプリを閉じて開き直しても、解説2の `getItem` で同じ名札から読み出して、前回の入力を復元できます。
- `useState` の値はアプリを閉じると消えますが、`AsyncStorage` に保存した値は**閉じても残る**——この違いが重要です。

> **用語: 永続化（えいぞくか）** とは「アプリを閉じても消えないように、データを残しておくこと」。`useState` は一時的な記憶、`AsyncStorage` は永続的な記憶、というイメージで使い分けます。

FlatList の応用 — グリッド・横スクロール・空のとき

> **▼ このコードがやること（先に日本語で）**：4.2で学んだ `FlatList` を、もっと実践的に使う3つの応用例です。① `numColumns` で**何列かに並べるグリッド表示**、② `horizontal` で**横方向にスクロールする並び**、③ `ListEmptyComponent` で**データが空っぽのときに案内文を出す**——どれもプロパティを1つ足すだけで実現できます。よくあるアプリの見た目がこれで作れる、という引き出しを増やしておきましょう。各指定の意味はコード内コメントにあります。

いつ使う・なぜ必要？ 写真アプリのような格子状の並び、おすすめ作品の横スクロール、検索結果ゼロ件のときの「該当なし」表示——どれも実際のアプリでよく見る形です。`FlatList` の基本（`data` / `renderItem` / `keyExtractor`）に**プロパティを足すだけ**で作れます。

コピー

```
import { FlatList, View, Text, StyleSheet } from "react-native";

const books = [
  { id: "1", title: "本A" },
  { id: "2", title: "本B" },
  { id: "3", title: "本C" },
  { id: "4", title: "本D" },
];

export default function FlatListAdvanced() {
  return (
    <FlatList
      data={books}
      keyExtractor={(item) => item.id}
      // numColumns : 2列のグリッドで並べる
      numColumns={2}
      // ListEmptyComponent : dataが空のときに表示する中身
      ListEmptyComponent={
        <Text style={styles.empty}>本がありません</Text>
      }
      renderItem={({ item }) => (
        <View style={styles.cell}>
          <Text>{item.title}</Text>
        </View>
      )}
    />
  );
}

const styles = StyleSheet.create({
  cell: {
    // 1列ぶんの幅を等分する
    flex: 1,
    margin: 6,
    padding: 20,
    backgroundColor: "#f1f5f9",
    borderRadius: 8,
    alignItems: "center",
  },
  empty: { textAlign: "center", color: "#94a3b8", padding: 24 },
});
```

横スクロールにしたいとき: 上の例の `numColumns={2}` を外し、代わりに `horizontal={true}` を付けると、リストが**横方向にスクロール**する並びになります（おすすめ作品の横並びなどに使います）。`numColumns` と `horizontal` は同時には使えないので、どちらか一方を使います。

▼ コードを1つずつ分解して解説

「FlatListに足す3つの便利プロパティ」を、1つずつ見ていきましょう。

解説1: 何列かに並べる `numColumns`

コピー

```
// numColumns : 2列のグリッドで並べる
numColumns={2}
```

- `numColumns`（ナム・カラムズ＝列の数）は、「**リストを何列のグリッド（格子状）で並べるか**」を指定します。
- ここでは `2` なので、項目が**2列ずつ**並びます。4件あれば「2列×2行」の格子になります。
- グリッドにするときは、各セル（`styles.cell`）に `flex: 1` を付けて「1列ぶんの幅を等分」させるのが定番です。これで列幅がそろいます。
- 写真の一覧や、本の表紙を格子状に並べたいときに使う、定番の指定です。

> **用語: グリッド表示** とは、項目を「縦横の格子状」に並べる見せ方のこと。写真アプリのサムネイル一覧のような並びをイメージしてください。

解説2: 横スクロールにする `horizontal`

コピー

```
// numColumns を外して、代わりにこれを付ける
// horizontal : リストを横方向にスクロールさせる
horizontal={true}
```

- `horizontal={true}`（ホリゾンタル＝水平）を付けると、リストが**横方向に並び、横スクロール**できるようになります（既定は縦スクロール）。
- 「おすすめの本」を横に並べてスッスッと見せる、といった見た目に向いています。
- `numColumns`（縦のグリッド）と `horizontal`（横スクロール）は**同時には使えません。**どちらか一方を選びます。
- 横スクロールのときは、各項目に `width`（幅）を指定しておくと並びが安定します。

> **用語: 横スクロール** とは、指で左右にスワイプして続きを見る並べ方。縦に長いリストと違い、限られた高さの中で「もっとあるよ」と見せたいときに使います。

解説3: 空のときに案内を出す `ListEmptyComponent`

コピー

```
// ListEmptyComponent : dataが空のときに表示する中身
ListEmptyComponent={
  <Text style={styles.empty}>本がありません</Text>
}
```

- `ListEmptyComponent`（リスト・エンプティ・コンポーネント＝リストが空のときの中身）は、「**`data` が空っぽのときに代わりに表示する中身**」を指定します。
- データが1件もないとき、何も出ないと利用者は「壊れた?」と不安になります。`ListEmptyComponent` で「本がありません」などの案内を出すと親切です。
- データがあるときは普通にリストが表示され、空のときだけこの中身が出る、と自動で切り替わります。自分で「空かどうか」を判定する必要はありません。
- 検索結果が0件のときの「該当する本はありません」表示などに、そのまま使えます。

> **用語: 空状態（からじょうたい / empty state）** とは「表示するデータが1件もない状態」のこと。ここに分かりやすい案内を出すかどうかで、アプリの親切さが変わります。

> **これらの先にあるもの:** ここで紹介した機能のほかにも、「画面の下からスライドして出るメニュー」や「複雑なタブ・ドロワー（横から出るメニュー）」などは、**React Navigation** という別のライブラリで作ります（Expo Routerもその上に作られています）。本書では深入りしませんが、「もっと凝った画面遷移がしたくなったら React Navigation を調べる」と覚えておけば十分です。

---

7. この章のまとめ

- React Nativeでは `View`（箱）・`Text`（文字、必ずこれで囲む）・`Image`・`TextInput`・`Pressable` などの専用部品を使う
- レイアウトは Flexbox（`flexDirection` / `justifyContent` / `alignItems` / `flex`）で決める
- スタイルは `StyleSheet.create` でまとめて管理する（第9章でNativeWindも導入）
- 大量のリストは `FlatList`（`data` / `renderItem` / `keyExtractor`）で表示する
- 画面遷移は Expo Router：`app/` にファイルを置けば画面になり、`<Link>` や `router.push()` で移動、`_layout.tsx` で全体の枠組みを定義
- `SafeAreaView` でノッチやホームバーを避ける

次の章へ: 画面の作り方が分かりました。第5章では、本のデータを保存する場所＝**Supabase（データベース）** をセットアップします。DBの選択肢の比較も詳しく解説します。